UNIVERSITY OF HERTFORDSHIRE

School of Physics, Engineering, and Computer Science

MSc. Advanced Computer Science Masters Project

7COM1039-0509-2022

Using NLP to detect Fake News by building a text-based News Classifier

Project Report

STUDENT ID NUMBER: **21027446**

NAME: **Nitesh Sah**

SUPERVISOR NAME: **Efosa Osagie**

21027446

MSC Final Project Declaration

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Data Science and Analytics at the University of Hertfordshire (UH).

It is my own work except were indicated in the report.

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on the university website provided the source is acknowledged.

Abstract:

News is one of the most basic things that humans need in their daily lives, and they receive news either from TV channels, social media platforms, or from newspapers and many more places. The timeliness and widespread accessibility of social media have contributed to its popularity as a source of news and information in today's world. Nevertheless, it gives rise to plenty of significant concerns. The issue of fake news requires immediate resolution. Talking about many threats, bogus news is one of the challenging threats and is quite harder to detect than other malicious threats. Research has been conducted and found that machine learning models are quite effective in detecting such news and giving good results. In this research couple of machine learning models have been applied to check the accuracy of the model and later decide which one is the best model to detect such news. Logistic Regression (LR) machine learning model has been proposed to detect fake news. By applying this model in the system, the accuracy of the news is 98.6%. The support vector Machine (SVM) model has also been applied to the system where it gives an accuracy of 99.30%. Hence, by comparing different metrics we conclude SVM is the best model for the used datasets.

Keywords: Machine Learning, Natural Language Processing (NLP), Data Pre-Processing, Logistic Regression (LR), Support Vector Machine (SVM), Accuracy.

Contents

Table of Figures:

Chapter I

1. Introduction:

The amount of fake news stories has been rising quickly. Although it is not a brand-new issue, it has recently seen significant growth. According to the definition provided by Wikipedia, fake news refers to information that is presented as news but is either incorrect or misleading in nature. Finding fake news has been a difficult and complicated undertaking. Humans have been found to have a propensity to believe false information, which makes the dissemination of false information much simpler. According to statistics, just 54% of people can identify fraud on their own without extra support. Fake news is hazardous because it readily deceives individuals and stirs up confusion within a community. This could also have negative effects on society.

Numerous surveys have revealed that individuals are increasingly using social media as their news source. Social media spreads news and information more quickly and widely than conventional media like TV and newspapers because of its timeliness and convenience. Meanwhile, false information makes use of the features of social media to propagate quickly and cheaply. Fake news is defined as information that is knowingly false and provable. This material is enhanced with doubtful facts and falsehoods, which contributes to social panic and interpersonal anxiety. The public's trust is destroyed by this incorrect information, which also has a negative impact on the economy and important political processes like elections and the stock market. (Samrudhi Naik, 2021)

Artificial intelligence-based systems based on natural language processing and machine learning have been employed to solve the false news identification problem. These automated techniques efficiently halt the spread of false and anomalous news because of advancements in artificial intelligence and technology. To further future attempts, these strategies have sparked a great deal of interest among researchers in recognising bogus news. (Noshin Nirvana Prachi, 2022)

1.1. Problem statement:

News is one of the most important things in human daily life. Nowadays, it is quite difficult to determine whether the information that is spreading is accurate because so many people choose to get their news from online sources and users of social media spend most of their time on these online platforms. People can acquire most of the information they need through these channels because it does not cost anything and can be accessed at any time from any site or from any location. Also, they can hear it from their friends or family. Since anyone can publish such content and no one is held responsible for its transmission, its veracity is undermined in comparison to more traditional methods of receiving information like reading newspapers or consulting a trusted source.

Fake news presents a risk to society because of its ability to easily confuse individuals and to build up uncertainty among groups. This makes fake news a particularly dangerous form of information. This may have extra adverse effects on society. The spread of false data unavoidably results in the circulation of rumours, which, in turn, may have an adverse impact on the people who have been victimized. We need a system that can find fake news quickly and efficiently at a large scale by combining the most cutting-edge technologies in natural language processing, video analysis, and machine learning. Only will be able to figure out how to solve this problem after using machine learning models.

1.2. Proposed Solution:

To overcome the problem of fake news detection this project proposes a few models like Logistic Regression (LR), and Support Vector Machine (SVM). These algorithms are particularly effective at capturing complex links within feature spaces, making them ideal for picking up on telltale linguistic and literary clues that point to the presence of false content. These algorithms can distinguish between real and fake news because of the careful feature engineering obtained through various techniques of word embedding to obtain numeric features for competition to identify patterns and make predictions. Some of the techniques includes TF-IDF, Word2Vec, GloVe, etc. The quality of feature engineering and the choice of features play a significant role in their performance.

1.3. Aims

The aim of this project is to construct various models utilizing machine learning techniques to determine the authenticity of news articles. Additionally, the project aims to assess and compare the performance metrics of these models, with the intention of assisting their possible use in future endeavours.

1.4. Objectives:

The objectives of this project are:

1. Conduct a literature review on subject of natural language processing (NLP) and fake news identification. To fully comprehend the state of the art in fake news detection and NLP approaches, this will include examining relevant academic books, articles, and papers.

2. Investigate several NLP methods to preprocess and extract relevant information from the text input. Examine whether features or representations, such as word embeddings or the TF-IDF, are most effective at detecting fake news when used with machine learning models.

3. Compare the performance of various false news detection models. Evaluate each algorithm using standard evaluation measures including accuracy, precision, recall, and F1-score, and then pick the one that produces the best overall results. Further, we must select suitable algorithms by evaluating their performance.

4. Identify the specific measures that will be used to judge the performance of the models. Accuracy, precision, recall, F1-score, receiver operating characteristic area under the curve (ROC-AUC), and confusion matrices are all common measures of performance.

5. Completely document the research process, beginning with the preprocessing of the data and ending with the results of the model, and must write a research report that is both understandable and instructive.

Overall, the objective of this project is to build a fake news detection model that can easily detect the news is true or false.

1.5. Research Questions:

Before starting the project, I did a lot of research where I could get some questions related to my thesis topic i.e., Fake News Detection Using NLP. Because of this project, I was able to dive deep into the topic of how Natural Language Processing algorithms are used in the development of fake news identification. Here are a couple of questions, that is outlined below:

1. What is the performance comparison amongst the state-of-the-art models in detecting fake news using NLP methods?
2. How can NLP techniques be used to design a text classification pipeline to detect fake news on social media?

1.6. Legal, Ethical, Social, and Professional Issues:

To develop or deploy a fake news detection system, it is very important to consider legal, ethical, social, and professional issues to get effective and responsible solutions.

The datasets which have been gathered for this project don't contain any personal information of any user or any sites.

Legal Issues: It is one of the most common and valuable issues for the research paper. The data or properties which have been collected for training purposes of fake news detection must be respected properly and all the data have been protected so that it doesn't harm their privacy.

Ethical Issues: Another one is an ethical issue, where the decision is made and shows a clear explanation of the data to the users. In a fake news detection system, trying to minimize false positives and false negatives helps to avoid harm to innocent parties and prevent them. The data which has been gathered for testing purposes are safe and it does not harm anyone's privacy.

Social Issues: In the same way, social issue plays a vital role in fake news detection and is closely linked to social media platforms. The spread of fake news creates chaos among the people, media, institutes and which led them to do some unnecessary things. The social issue of fake news circulates around and has a high impact on

society and individuals. To stop this data must be filtered properly so it doesn't harm their feelings and sentiment.

Professional Issues: Talking about the professional issue in fake news detection includes a lot of challenges and responsibilities for researchers, developers, and organizations in documenting and deploying the detection system. Users and the other parties should have a clear vision of how the system works, and how it shows the content. Sensitive information must be kept private and kept away from the leak, which is to be removed or anonymized so that only authorized individuals can access it.

1.7. Structure of the Report:

The report's format explains an overview of the research. There are many chapters in this report, which makes it simple to find specific information. The report's format is outlined in brief below:

1. **Introduction**

To start the report, the Introduction part is compulsory. This section of the report represents the initial segment. This section offers a comprehensive explanation of the process involved in detecting fake news. Likely Problem domain and proposed solution followed by the introduction part. This section also includes the aims and objectives of the report and a couple of research questions. In this chapter, the last section discusses several professional and social issues.

2. **Literature Review**

In this section, different approaches and related works have been discussed in detail. The review of different papers helped me to get the idea of developing fake news detection and the summary of different papers is provided in this chapter 2.

3. **Methodology and Technologies**

This is the chapter 3 of this report. This chapter includes a comprehensive explanation of the methodology and dataset used in this project. In this chapter different tools, programming languages, software used, and the libraries used in this project have been discussed. Similarly, many machine-learning algorithms have also been described.

4. **Implementation**

Chapter 4 is followed by Chapter 3 where different code has been explained in brief with their screenshots. Chapter 4 describes the initial data analysis, data pre-processing, and data cleaning. This chapter is very important for easy understanding of the code.

5. **Results and findings**

In this chapter, different algorithm's accuracy has been described with their following snapshots.

6. **Conclusion and Future works**

This is the last chapter of this report. In this section, the conclusion of the report and future related work has been discussed.

Chapter II

2. Literature Review

2.1. Machine learning methods on fake news detection:

A survey conducted by (Singh & Patidar, 2022) on fake news detection using machine learning methods where Various sequential classifiers, including Hawkes algorithms, linear conditional random fields (CRF), long short-term memory (LSTM), and tree CRF, have been employed in the classification of rumour positions on social media platforms. Among these classifiers, LSTM has demonstrated superior performance compared to the others. Artificial intelligence methods like K-Nearest Neighbour, Stochastic Gradient Descent, Decision Trees, Linear Support Vector Machines, and Support Vector Machines have been used to detect fake news, with LSVM having the highest precision of 92%. Data imputation strategies, namely the utilisation of data modelling for missing values, have been employed to augment the identification of incorrect data. Notably, the implementation of Multi-Layer Perceptron (MLP) classes has demonstrated a notable enhancement in accuracy, resulting in a 16% increase.

The researchers used different machine learning algorithms like KNN, Naïve Bayes, Random Forest, XGBoost, and support vector machine to identify false news by analysing news articles, postings, and tales. XGBoost performed better with an accuracy of 0.86 (Reis, et al., 2019). Natural Language Processing (NLP) techniques and Machine Learning (ML) algorithms, such as naïve Bayes and Support Vector Machines (SVM), have been utilised in the detection of misinformation disseminated on social media platforms. These approaches have achieved respective accuracy rates of 63% and 75%. The report additionally examines potential strategies for reducing the spread of misinformation and identifies possible areas for future research in this domain.

2.2. Using Deep Learning and Natural Language Processing to detect fake news

The objective of this research paper is to examine the efficiency of a combined deep learning and natural language processing (NLP) model in the detection of false news articles. The proposed approach suggests the integration of deep learning and natural language processing (NLP) in a hybrid model, aiming to get a significant level of clarity

in the detection of fake news. The author in this research paper clarifies the difficulties associated with the manual identification of false material and the complex processes of assessing the accuracy of content inside a story. The statement underscores the necessity of an automated system that possesses the capability to detect and identify fake news with a high degree of effectiveness and accuracy. The purpose of this work is to describe the process that was used to construct an identification system for fake news utilising a Word2vec model in conjunction with a Long Short-Term Memory (LSTM) model. The system is trained and evaluated with the use of two distinct dataset collections, one comprising actual news datasets and the other consisting of false news datasets.

This study examines the correlation between three distinct variables, namely the number of training cycles, data diversity, and vector size, and the levels of accuracy exhibited by the system. The study reveals that these variables produce a statistically significant impact on the accuracy of the system. And at last, research concludes that the combination of LSTM and Word2vec models into a single platform may effectively identify fake news with a high degree of accuracy, obtaining a minimum predicted accuracy level of 90% in the process (Anand & Burra Venkata Durga, 2022)

2.3. Fake News Detection using non-ML methods:

There exists a limited number of research that have created rule-based models for the purpose of detecting fake news. In their study researcher developed a series of criteria aimed at detecting instances of COVID-19 disinformation in Arabic tweets. The criteria primarily focused on identifying patterns of exaggeration and emotional content within the tweets. The approach employed exhibited a high level of precision. Hybrid model was introduced which utilises established criteria to classify news stories as either authentic or counterfeit. The criteria incorporated domain knowledge derived from fact-checking websites. The approach demonstrated superior performance compared to machine learning algorithms when applied to a limited dataset. These experiments provide evidence of the capacity of rule-based systems to accurately identify false information without the need for extensive training data. Further investigation is required to explore the application of rule-based approaches in circumstances and languages that have received limited attention in current research. However, it is evident that rule-based models exhibit initial potential as a viable approach to address

the issue of disinformation when faced with resource constraints (Alotaibi & Alhammad, 2022) .

2.4. Fake news detection using metrices evaluation:

Logistic Regression and Compare Textual Property with Support Vector Machine to detect fake news by (N. Leela & M., 2022). In this research, they gathered the datasets and after that they tested and trained both data sets i.e., real, and fake datasets where they used the Logistic regression (LR) algorithm and Support Vector Machine (SVM) algorithm to show accuracy based on news data. In the proposed research, LR and SVM algorithms were compared for their accuracy and precision in detecting bogus news. On the false news dataset, the LR method appeared to have greater accuracy (95.12%) and precision (93.62%) than the SVM algorithm (91.68%) and lower accuracy (89.20%). When comparing the LR algorithm to the SVM method, the LR algorithm was shown to be somewhat superior in its ability to detect fake.

The purpose of this fake news detection is that it will detect fake news so that the users will be only reading real and accurate news, not false news. I have gone through various research papers and found out many things from those papers and got a lot of ideas from that research. Using NLP techniques and machine learning algorithms to detect fake news. There are several machine-learning models which can be used to detect fake news and provide accurate news. And, in this paper, I have used Logistic Regression and Support Vector Machine models to detect fake news. After using both models we will get the accuracy of each model and get the result. The one which gives the best accuracy will be the best model for detecting fake news.

Chapter III

3. Methodologies and Technologies:

3.1. Workflow Diagram:

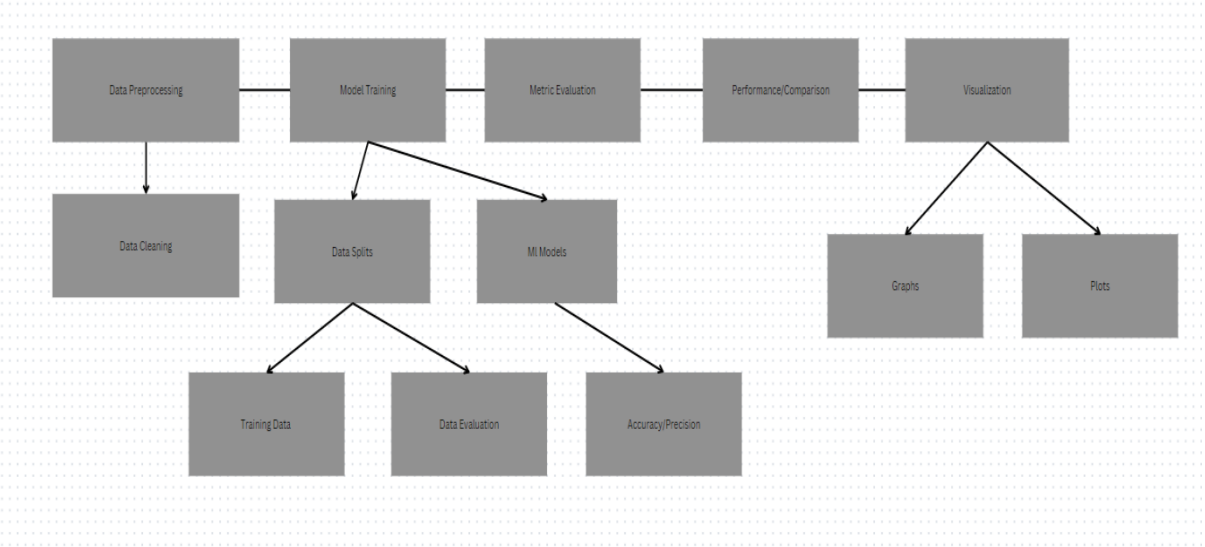

Figure 1 Workflow diagram

Python:

There are various programmes and languages that could be used for data extraction, processing, and visualisation. Python, R and Julia are the most popular programming languages. As python is most demanding and very familiar language so for this research python has been used. Both R and Julia are high-performance languages well suited to large-scale, statistically heavy operations; R is a specialised language with extensive support for statistics and graphics. The best fit for this research project is python programming language.

It is a high-level programming language and is open source which is quite easy to read and write. Guido Van Rossum was the creator of this language in late 1980s with the vision of simple syntax, easy and clear to understand for both the beginners and experts. As it is one of the versatile languages, which allows users to create a variety of multiple projects like small to large applications. The programs written in python runs on different operating systems like macOS, and Linux because of its cross-platform nature which do not need any major changes. (Pacheco, n.d.)

Python is an excellent choice for data-driven tasks like predictive modelling and pattern recognition because it also has several machine learning packages, such as sci-kit-learn and TensorFlow. Python is widely utilized in many areas of software development, including web development, scripting, and many more, in addition to data analysis. It is a well-liked option for a variety of applications due to its adaptability, simplicity, and sizable development community. Python has a large and active developer community, resulting in a diverse range of modules and frameworks for data manipulation, visualization, machine learning, and web development. (Rohan Chopra, 2019)

Popular Python data analysis packages include NumPy for numerical computation, Pandas for data administration and analysis, and Matplotlib for plotting. The following libraries and packages are discussed below:

1. **NumPy**

It stands for "Numerical Python," which is the full name of the word NumPy. It's a python package that makes working with large multidimensional arrays faster and easier. It is possible to execute a vast variety of mathematical operations with these arrays (McKinney, 2017).

2. **Pandas**

It provides data structures and functions at a high level that make working with structure or tabular data fast, simple, and expressive. It is built on top of NumPy, and because data manipulation, preparation, and cleaning are crucial to data analysis, pandas is one of the most essential tools for these tasks (McKinney, 2017).

3. **Matplotlib**

Matplotlib is the most popular python library which is used for producing two-dimensional plots of array. Matplotlib is a cross-platform data visualisation package that is constructed using NumPy arrays and specifically developed to function smoothly with the wider SciPy stack. The introduction of this concept was attributed to John Hunter in 2002. Matplotlib includes a diverse array of plot types, including line plots, bar plots, scatter plots, histograms, and more (KattamuriMeghna, n.d.).

4. **NLTK**

When it comes to text analysis and natural language processing in Python, this package is among the best available. The Natural Language Toolkit (NLTK) is a popular Python library for working with and analysing text written in the human language. The software provides intuitive navigation to over 50 dictionaries and corpora, including WordNet. It also has a set of text processing packages that can do operations like classifying, tokenizing, stemming, tagging, parsing, and semantic reasoning. Additionally, it provides APIs to industry-standard, highly effective natural language processing libraries (NLTK, 2023). This project employs the Natural Language Toolkit (NLTK) to do several tasks including tokenization, removal of stopwords, lemmatization, and preprocessing of words.

5. **Seaborn**

Seaborn is an extension of the matplotlib library that provides a Python library for data visualisation. The software provides an intuitive interface that facilitates the creation of visually appealing and useful graphs and charts. The platform provides a variety of visualisation options, such as scatter plots, line plots, heat maps, and distribution plots. Python is a powerful and adaptable instrument for the purpose of visualising and exploring data (Pooja, 2023)

6. **Regex**

The word "Regex" is a short form for "Regular Expression". The 're' package in Python is utilised for conducting regular expression searches. Regular expressions are a highly potent instrument for identifying patterns within textual data and are frequently employed for purposes such as text search and replacement, input validation, and extraction of information from extensive datasets.

7. **Scikit-Learn:**

It is a Python machine learning package also known as sklearn. Sklearn is a fantastic tool for learning about machine learning because it is easy to use, available for free, and can be implemented in business settings. Data modelling, as compared to loading, manipulating, visualising, and summarising, is Scikit-Learn primary focus (Paper,

2019). A machine learning model for distinguishing between fake and genuine news is constructed and trained using this library.

Jupyter Notebook:

When performing data analysis in Python, many different integrated development environments (IDEs) are available and are frequently utilised. PyCharm, Spyder, and Jupyter Notebook are a few of the tools that fall into this category. For this project, I am employing Jupyter Notebook.

It is one of the most essential tools for data scientists that work with the Python programming language. This is because they provide an ideal environment in which to construct data analysis workflows that are replicable. The data can be loaded, modelled, and changed all inside the limits of a single notebook, making it simple and quick to experiment with code and investigate various concepts. The Jupyter notebook is one of the most popular options for generating, editing, and running notebooks. It is also extremely extensively documented and features an intuitive user interface. The notebook operates as a web application known as the "Dashboard" or "control panel," which displays local files and enables users to access notebook papers and run snippets of code (Galea, 2018).

The Jupyter Notebook is widely utilised within the Data Science community for Python-based data exploration and analysis. The selection of Jupyter notebook has been made for this project. The platform facilitates the installation of libraries, the loading and preprocessing of data, and the training and preparation of the classifier.

3.2. Dataset

The dataset is a fundamental component of data analysis since it serves as the primary source of data for analytical purposes. The effectiveness of the analysis and the conclusions that may be taken from it are directly influenced by the quality of the dataset. If the dataset is incorrect insufficient, or defective, it may hamper the analysis and undermine the reliability of the conclusions. The magnitude of the dataset is a crucial factor to consider. A larger dataset has the potential to provide more comprehensive insights. The study of such a dataset also requires a greater investment of time and money. In contrast, a smaller dataset may lack comprehensiveness but offers the advantage of quicker evaluation and requiring fewer resources. The structure of the dataset is also of extreme significance. The

arrangement of the data should be designed in a manner that promotes convenient analysis and interpretation. To ensure the accuracy and consistency of the data, it may be necessary to engage in cleaning and preprocessing procedures.

Dataset Link:

https://www.kaggle.com/datasets/jainpooja/fake-news-detection/download?datasetVersionNumber=1

This data set is a collection of two dataset named: fake.csv and true.csv, which is a dataset obtained from open source, here Kaggle. This dataset contains hand-selected true and false news, with false news containing 23481 rows and 4 columns and true news containing 21417 rows and 4 columns with column values of ['title', 'text', 'subject', and 'date'].

To classify the fake and true news using a machine learning model we labelled the data set by using a column named "Label" where label 1 is assigned for True news and Label 0 is assigned for the fake news.

Preprocessing:

Since we are completely into text analysis, the columns title, subject and date are of low importance on our side hence we dropped those columns, now we are with column text (representing the news) and label for each dataset. The two data sets were concatenated along the rows. To ensure randomness, the merged rows are shuffled. It is ensured that no null value exists for the text column.

Various preprocessing techniques were used to allow the model to focus on the semantic content of the text. Some of them are:

Noise reduction:

Words that contain digits on them and text enclosed in square brackets are removed, along with URL, html tags, non-word characters and punctuations using different regexes. This helped to remove the noise and make the text cleaner and more suitable for analysis.

Normalization:

The conversion of words to lowercase was implemented to decrease the dimensionality of the feature space.

Dimensionality reduction:

Stopwords were removed with the help of NLTK library which helped to reduce the dimensionality of the text data.

Train test split:

The cleaned dataset was split into x_train, x_test, y_train and y_test where X represents the text(news), and Y represents the label. 70 % data was used for training and 30% was used as test data. Specific random state was defined to ensure the reproducibility of the machine learning model.

Feature extraction:

The TF-IDF (Term Frequency - Inverse Document Frequency) vectorization method was used to turn unorganised text data into numerical features. The TF-IDF numbers for each term were found by increasing its TF (Term Frequency) value by its IDF (Inverse Document Frequency) value. Term Frequency (TF) is a measure that shows how often a certain word or phrase is used in a text. It gives more weight to words that are used more often in the text. Inverse Document Frequency (IDF), on the other hand, is a way to figure out how important a word is in the whole collection. It is found by dividing the logarithm of the total number of documents by the number of documents that contain the term. IDF gives more weight to words that are used less often in a text. The process of feature extraction was used on both the training dataset and the testing dataset.

3.3. Initial Data Analysis:

For the initial Data analysis, there is a bar chart which has been generated using a matplotlib where it visualizes the distribution of labels in datasets. There are two vertical bars displayed in the chart. One bar is in red colour which represent “True” label (1) and other is “Fake” label (0) which is coloured green. Numerical annotations would be placed above each bar to indicate the count of data points corresponding to each label category. The inclusion of these annotations would offer a comprehensive

depiction of the number of occurrences that have been categorised as "True" and"Fake."

There are two datasets in this project i.e., fake dataset and real dataset. WordCloud

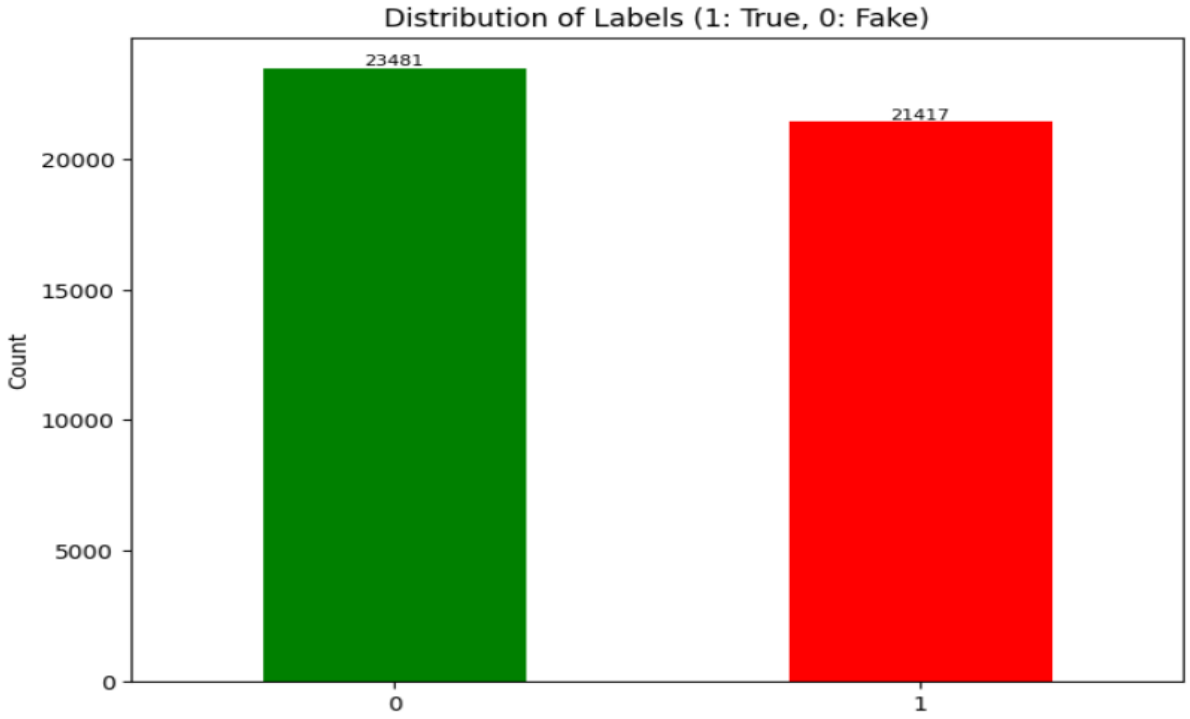

Figure 2 Distribution of data

library of python is used to generate the output of top common 500 words used in both the datasets.

Figure 3 500 Common words in True dataset

Figure 4 500 Common words in fake dataset

3.4. Fake news detection using logistic regression:

Some prior work has been done on the topic of false news identification across several platforms, and it takes advantage of the platform's natural capacity for logistic regression to predict text classifications. Logistic regression's predictive strength and unpredictability values make it a useful tool for dealing with binary classification problems. Short input texts are a strength of the logistic regression detection model, and respectable levels of accuracy can be obtained. (Shivani, et al., 2023).

After preprocessing and cleaning of the data, we implemented logistic regression, a widely used binary classification algorithm. The goal of logistic regression is to predict the likelihood of an event occurring given a set of independent factors (Anon., n.d.).

In this study, TF-IDF vectorization results were the dependent variable whereas label was the independent variable. We used the scikit-learn library in python to build our model. The logistic regression model calculates the probability of news articles being fake or real based on the extracted features. The LR model was applied to our training data and metrics such as accuracy, precision, recall, F1-score, and AUC-ROC was used to measure its performance.

Logistic Regression:

Logistic regression is a strong analytical technique that is particularly useful when the dependent variable is binary in nature (Peng, et al., 2002).

The dependant variable is restricted to values between 0 and 1 because the result of the logistic regression is a probability. The logistic or sigmoid function is used to model the probability in logistic regression. When used as an activation function, the sigmoid function introduces non-linearity into a machine learning model.

Here, the sigmoid function is defined as.

$$sig(t) = 1/ (1+e-t)$$

The result of the sigmoid function for some input t is written as sig(t). An input t is transformed by the sigmoid function into the range [0,1]. A predicted value of 1 is expected if t tends towards the positive infinity range, while an anticipated value of 0 is expected if t tends towards the negative infinity range. If the sigmoid function's output is larger than 0.5, it will be classified as class 1 (in this case, true news), and if it is less than 0.5, it will be classified as class 0 (false news). Input to the logistic regression will be the linear combination of the features.

The following figure show the sigmoid curve:

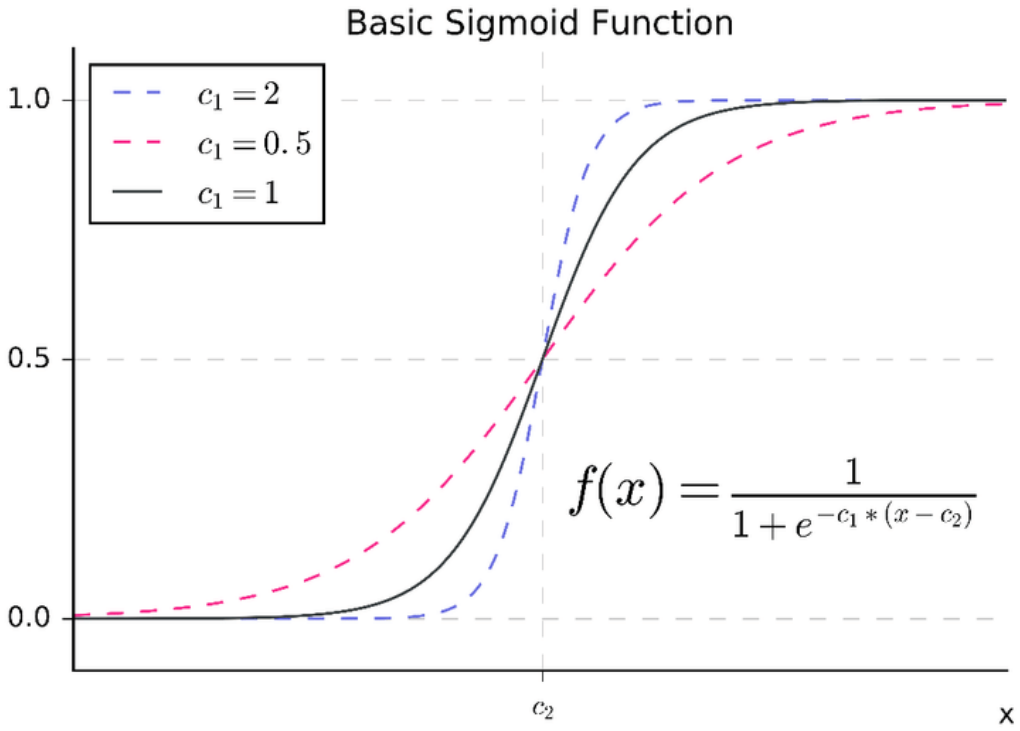

Figure 5 Basic Sigmoid Function with two Parameters

3.5. Fake news detection using SVM

SVM is an algorithm used in machine learning. In SVM, hyperplanes are used to categorise data points. In SVM, the hyperplane is found with the use of the SVM algorithm. When searching for hyperplanes in a space with N dimensions and N characteristics, the SVM technique is typically employed. The number of features is what sets the size of the hyper plane.

(Leela Siva Rama Krishna & Adimoolam , 2022).

The SVM classifier was imported from scikit-learn and an instance of it was created. For we don't know the data patterns of the dataset, we use the rbf kernel in SVM. It is a non-parametric model that works well with non-linear and high-dimensional data. We fit the model to the pre-processed training dataset and predictions were made for the test dataset. Metrics like accuracy, precision, recall, F1-score, and confusion matrix were evaluated.

Support Vector Machine

The Support Vector Machine (SVM) is a linear classifier technique that use a decision plan to establish decision boundaries. The procedure involves the construction of an ideal hyperplane, which is a multidimensional border, to effectively partition data into distinct classes while also maximizing the margin or space between them. The determination of these important limits is based on a specific subset of data points known as support vectors. The careful choice of a suitable kernel function is crucial in achieving optimal separation and minimizing mistakes, especially in the context of non-linear data.

Although SVM was originally intended for binary classification, it can be extended to address multiclass problems by decomposing them into numerous binary classification subproblems. When confronted with non-linear data, a common strategy is to employ a kernel function that maps the data into a higher-dimensional space, enabling the successful discrimination of patterns using a hyperplane. Frequently employed kernel functions encompass the Linear, Polynomial, Radial Basis Function (RBF), and Sigmoid functions (Scholkopf & Smola, 2018).

SVM's advantage lies in its ability to mitigate overfitting, which occurs when the separation function becomes overly complex. This is achieved by giving less importance to data points that are far from the separation boundary. Once the optimal hyperplane is established, most of the data points, except for the support vectors, are considered redundant.

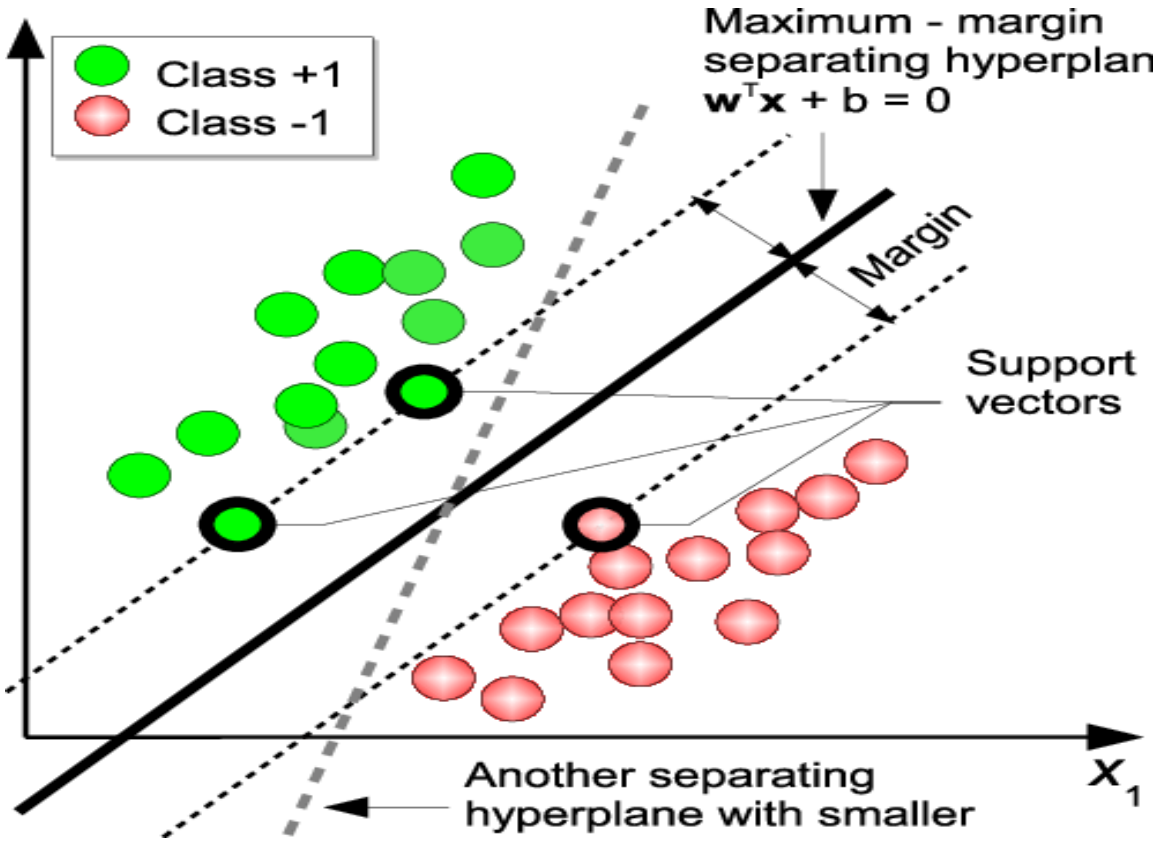

Figure 6 Principles of Support Vector Machines

Chapter IV

4. Implementation:

Imports:

The provided code snippet appears to be the initial part of a typical implementation section in a machine learning or natural language processing (NLP) project. It includes necessary imports and setup steps for data preprocessing and model development.

```
[1]: import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
     import nltk
     import tensorflow
     from nltk.corpus import stopwords
     from sklearn.utils import shuffle
     from sklearn.model_selection import train_test_split
     from sklearn.metrics import accuracy_score
     from sklearn.metrics import classification_report

     import re
     import string
```

Figure 7 Importing Libraries and Modules

Read the file containing fake and true news:

```
[2]: df_fake = pd.read_csv('./Fake.csv/Fake.csv') #Fake dataset

[3]: df_true = pd.read_csv('./True.csv/True.csv') #True Dataset
```

Figure 8 Loading datasets

The data in the CSV files loaded into pandas DataFrames, named “df_fake” and “df_true”.

Check the columns of the dataset

```
[4]: df_true.columns

[4]: Index(['title', 'text', 'subject', 'date'], dtype='object')

[5]: df_fake.columns

[5]: Index(['title', 'text', 'subject', 'date'], dtype='object')
```

Figure 9 Checking datasets columns

Checking the column of both datasets.

Check the shape of both dataset

```
[7]: df_true.shape

[7]: (21417, 4)

[8]: df_fake.shape

[8]: (23481, 4)
```

Figure 10 Obtaining the row and column of both datasets

We obtained that there are 21417 rows and 4 columns in the true dataset whereas there are 23481 rows and 4 columns in the false dataset.

Labelling the dataset as true and false (1 for true and 0 for false)

A new column named labelled is added to both the dataset, where 1 is used as value for the true dataset and 0 is used for the false.

```
[9]: df_fake['Label']=0

[10]: df_true['Label']=1
```

Figure 11 Giving the label of datasets

Dropping the insignificant columns

```
[16]: df_fake.drop(['title','subject','date'],axis=1,inplace=True)

[17]: df_true.drop(['title','subject','date'],axis=1,inplace=True)
```

Figure 12 Dropping Columns

Validate if there exists any null value in those dataset

```
[18]: # to check if there is any null in dataset
      df_true.isna().sum()

[18]: text     0
      Label    0
      dtype: int64

[19]: df_fake.isna().sum()

[19]: text     0
      Label    0
      dtype: int64
```

Figure 13 Checking the null value in both datasets

This ensures that there are not any null values in both attributes of fake and true dataset.

Merging the dataset

```
[23]: merged_data = pd.concat([df_fake,df_true],axis=0).reset_index(drop=True)
```

Figure 14 Merging the datasets

The above code snippet concat the two dataframe row wise into a single dataframe named merged.

Reshuffling the dataset

```
[72]: # let's reshuffle the dataset we have concatenated the fake and real news(good practice)
      merged_data = shuffle(merged_data,random_state=11).reset_index(drop=True)
```

Figure 15 Reshuffling the merged datasets

The above code reshuffles the merged dataset, which is a good practice. Random state was used to maintain the reproducibility.

Preprocessing:

```
[27]: stop_words = set(stopwords.words('english'))
def process_text(text):
  text=text.lower()
  text = re.sub('\w*\d\w*', '', text) #remove those words that contains digits
  text = re.sub('https?://\S+|www\.\S+', '', text)  #remove urls
  text = re.sub('\[/*?\]','',text) #remove text enclosed in sq bracket, this removes references if exists
  text = re.sub('<.*?>+', '', text) #Removes HTML tags
  text = re.sub("\\W"," ",text) #replace non-word characters with spaces
  #remove punctuation
  text = text.translate(str.maketrans('','',string.punctuation))
  text = ' '.join(word for word in text.split() if word not in stop_words)
  #since we are more concerned on analyzing the textual content for classifying fake news , we ought to remove the hyperlinks on them.
  return text
```

Figure 16 Preprocessing the datasets

The provided Python code defines a process_text function that performs essential text preprocessing tasks for natural language data. It starts by initializing a set of common english stopwords to eliminate frequently occurring words that carry little meaning. The text is then converted to lowercase to ensure uniformity. Subsequently, the function removes words containing digits, URLs, square bracket-enclosed text, HTML tags, and non-word characters. Punctuation is also stripped from the text.

The code concludes by filtering out the stopwords, ensuring that only the meaningful textual content remains. This preprocessing is especially valuable for fake news detection tasks, as it prepares the text data by cleaning and normalizing it. It helps create a more accurate and efficient feature representation for machine learning models. By removing noise and irrelevant information, such as hyperlinks and non-alphanumeric characters, the function enhances the quality of the text data, making it suitable for classification and analysis.

```
[28]: merged_data['text']=merged_data['text'].apply(process_text)
```

Figure 17 Functions applied to clean raw text

The above function named “process_text” is applied to the attribute text so that the raw text is more clean and ready for further operation.

Definition of dependent and independent variables

```
[44]: #lets define the dependent and independent variables
      X = merged_data["text"]
      Y = merged_data["Label"]
```

Figure 18 Defining Dependent and independent variables

Splitting the dependent and independent variable into train and test

```
[45]: #splitting the data into train test set

[46]: x_train, x_test, y_train, y_test = train_test_split(X,Y,test_size=0.3,random_state=11)
```

Figure 19 Split the datasets into training and testing sets

The provided code segment partitions the datasets X and Y into separate training sets (x_train, y_train) and testing sets (x_test, y_test), allocating 30% of the data for testing purposes. Additionally, a random seed is established to ensure reproducibility, with a random_state value of 11.

Feature extraction

```
[47]: # conversin text to Vectors
      from sklearn.feature_extraction.text import TfidfVectorizer

[48]: vector = TfidfVectorizer()
      x_trainV = vector.fit_transform(x_train)
      x_testV = vector.transform(x_test)
```

Figure 20 Vectorizing using TF-IDF vectorizer

This code snippet uses scikit-learn's TfidfVectorizer to convert text data into TF-IDF (Term Frequency-Inverse Document Frequency) vectors. It does this for both the training (x_train) and testing (x_test) datasets.

Implementation of logistic regression

```
[49]: # implementation of logistic regression
      from sklearn.linear_model import LogisticRegression
      logistic_regression = LogisticRegression()
      logistic_regression.fit(x_trainV,y_train)

[49]: ▾ LogisticRegression
      LogisticRegression()
```

Figure 21 Implementing logistic Regression

This code snippet uses scikit-learn's Logistic Regression model to create a logistic regression classifier and then fits (trains) it on the TF-IDF transformed training data (x_trainV) with corresponding labels (y_train).

Prediction on test data

```
[50]: # lets do prediction on test data
      logistics_pred = logistic_regression.predict(x_testV)
```

Figure 22 predicting test data using trained logistic regression classifier

This code makes predictions using a trained logistic regression classifier (logistic_regression) on the test data (x_testV) and stores the predicted labels in the variable "logistics_pred." It computes what the model predicts for each data point in the test set.

Accuracy Measurement

```
[51]: logistic_regression.score(x_testV,y_test)

[51]: 0.9864884929472902
```

Figure 23 Accuracy score of a trained logistic regression classifier on the test data

This code calculates and returns the accuracy score of a trained logistic regression classifier (logistic_regression) on the test data (x_testV) using the corresponding true labels (y_test). The score represents the model's performance in correctly classifying the test data.

Classification report logistic regression

```
[52]: print(classification_report(y_test,logistics_pred))
```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	7130
1	0.98	0.99	0.99	6340
accuracy			0.99	13470
macro avg	0.99	0.99	0.99	13470
weighted avg	0.99	0.99	0.99	13470

Figure 24 Summary of various performance metrics of the classification model

The provided code snippet is responsible for generating a classification report, which is a concise overview of the various performance metrics associated with a classification model. The process involves evaluating the performance of a classifier, most likely a logistic regression model, by comparing the actual labels (y_test) with the predicted labels (logistics_pred). This evaluation includes the calculation of various metrics such as precision, recall, F1-score, and support for each class involved in the classification task. This approach provides a valuable means of thoroughly evaluating the performance of the model.

The provided classification report summarizes the performance of a binary classification model, likely trained on some data. It includes several key metrics that assess the model's effectiveness in distinguishing between two classes, labelled as 0 and 1.

Precision, which measures the accuracy of positive predictions, is exceptionally high for both classes, with 0.99 for class 0 and 0.98 for class 1. This indicates that the model makes very few false positive errors when classifying instances.

Recall, representing the ability to correctly identify all instances of the positive class, is also impressively high for both classes, with values of 0.99. This means that the model effectively captures nearly all actual positive instances.

The F1-score, which effectively balances precision and recall, demonstrates a commendable value of 0.99 for both classes, so showing a robust and powerful overall performance.

The "support" values indicate the number of instances in the test dataset for each class, with 7130 instances of class 0 and 6340 instances of class 1.

The model achieved an overall accuracy rate of 99% on the test dataset, indicating that it accurately classified the class for 99% of the cases in the test set.

The "Macro Avg" and "Weighted Avg" values both demonstrate excellent performance with scores of 0.99 across all metrics, indicating consistent and high-quality performance for both classes, even when accounting for class imbalance. In summary, this classification report showcases a highly accurate and reliable binary classification model.

Plot of confusion matrix and ROC curve

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, roc_curve, auc
true_labels = y_test
# Calculate the confusion matrix
conf_matrix = confusion_matrix(true_labels, logistic_regression.predict(x_testV))

# Plotting confusion matrix using seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Purples', cbar=False)
plt.xlabel('Predicted_Labels')
plt.ylabel('True_Labels')
plt.title('Confusion Matrix')
plt.show()


# Plotting ROC curve
plt.figure()
fpr, tpr, thresholds = roc_curve(true_labels, logistic_regression.predict_proba(x_testV)[:, 1])
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, color='darkgrey', lw=3, label='ROC curve (area = %0.3f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='skyblue', lw=3, linestyle='--')
plt.xlim([0.0, 1.05])
plt.ylim([0.0, 1.07])
plt.xlabel('False Positive Rate(FPR)')
plt.ylabel('True Positive Rate(TPR)')
plt.title('Receiver Operating Characteristic(ROC)')
plt.legend(loc="lower right")
plt.show()
```

Figure 25 Code for plotting confusion matrix and ROC curve

This code snippet is a Python script that utilizes the Matplotlib and Seaborn libraries, along with scikit-learns metrics functions, to generate and display two important evaluation plots for a binary classification model.

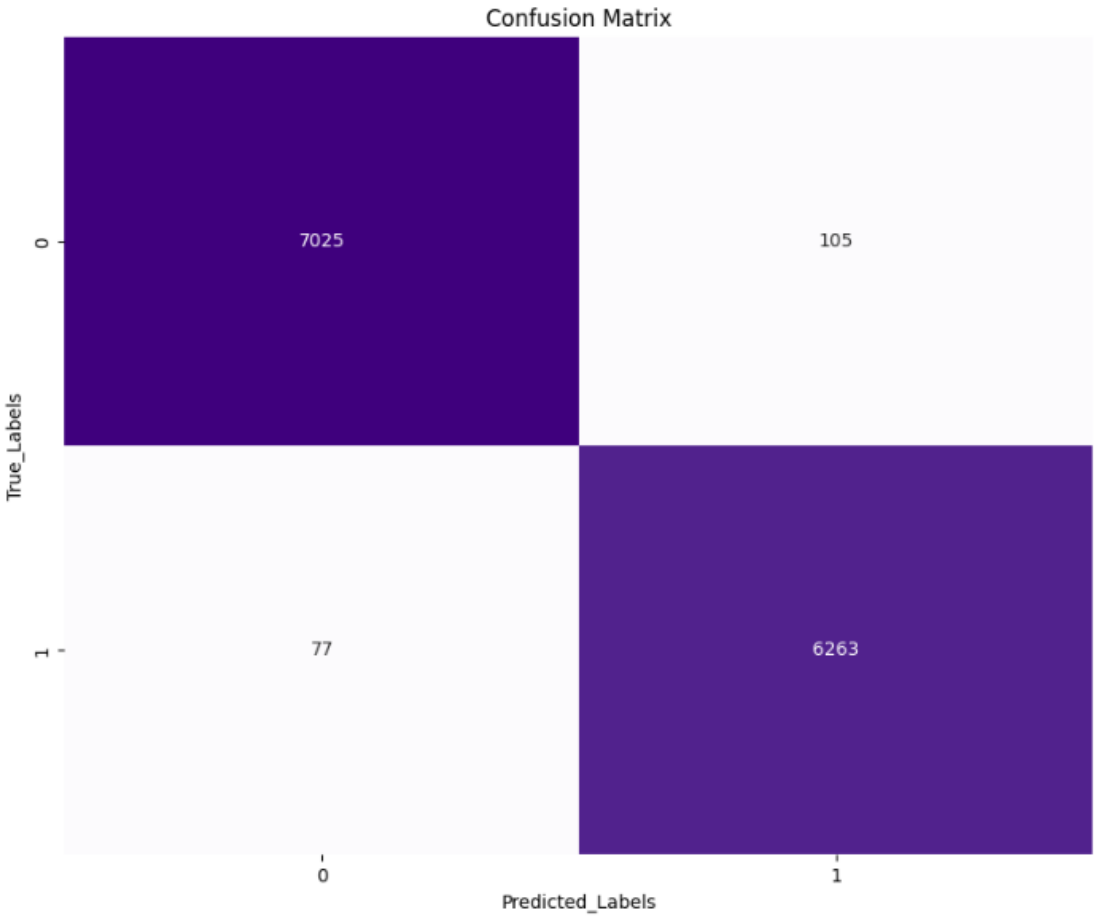

Figure 26 confusion matrix for Logistic Regression

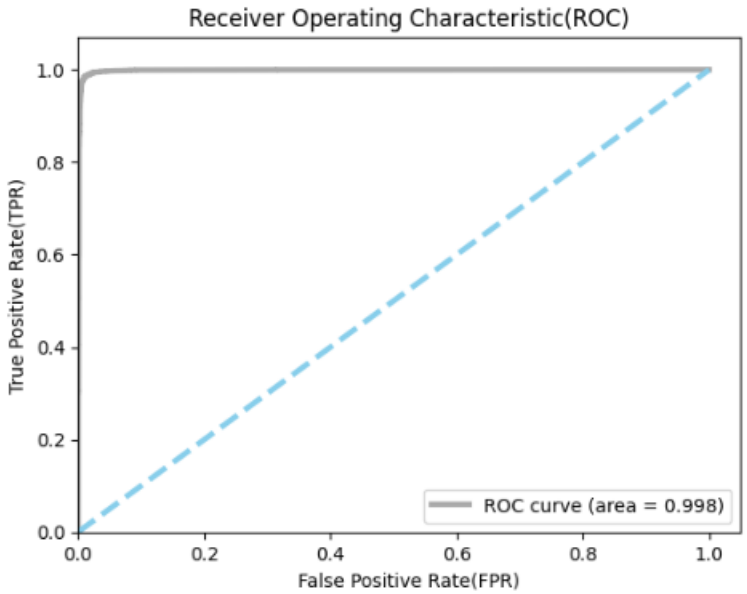

Figure 27 ROC Curve for Logistic Regression

Implementation of support vector machine

```
[57]: ############support vector machine###############
      from sklearn.svm import SVC

[58]: svm_model = SVC(kernel='rbf',random_state=0)
      svm_model.fit(x_trainV,y_train)

[58]: SVC
      SVC(random_state=0)
```

Figure 28 Implementation of Support Vector Machine

The above snippet imports the SVM classifier from scikit-learn's 'svm' module. Line 2 initializes a svm model. "**Radial basis function",** one of commonly used kernels which enables SVM to capture complex relationships in data. svm_model.fit(x_trainV, y_train): Line 3 trains (fits) the SVM model using the training data (x_trainV) and their corresponding labels (y_train). During training, the model learns to find the optimal decision boundary that separates the different classes in the feature space.

Prediction on test data

```
[67]: y_pred = svm_model.predict(x_testV)
```

Figure 29 Prediction on test data using a trained svm

This code makes predictions using a trained svm on the test data (x_testV) and stores the predicted labels in the variable "y_pred." It computes what the model predicts for each data point in the test set.

Accuracy measurement

```
[68]: svm_accuracy = accuracy_score(y_test,y_pred)

[69]: svm_accuracy

[69]: 0.9930957683741648
```

Figure 30 Accuracy score of a trained svm classifier on the test data

This code calculates and returns the accuracy score of a trained support vector machine classifier on the test data (x_testV) using the corresponding true labels (y_test). The score represents the model's performance in correctly classifying the test data.

Classification report SVM

```
[70]: print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	7130
1	0.99	0.99	0.99	6340
accuracy			0.99	13470
macro avg	0.99	0.99	0.99	13470
weighted avg	0.99	0.99	0.99	13470

Figure 31 Summary of various performance metrics of the classification model

The provided code sample generates a classification report, which is a short overview of several performance metrics for a classification model. The process involves the comparison of the actual labels (y_test) with the predicted labels (y_pred) generated by a classifier, often a logistic regression model. This evaluation provides many metrics, including precision, recall, F1-score, and support, for each class within the classification task. This approach provides a valuable means of thoroughly evaluating the performance of the model.

The classification report supplied presents a thorough assessment of the binary classification model's performance on a dataset comprising two unique classes, labelled as 0 and 1. Each row within the report corresponds to a certain class, and the report covers a variety of crucial performance criteria.

To begin with, precision is a metric that evaluates the correctness of positive predictions. Notably, the precision values for both class 0 and class 1 are exceptionally

high, reaching 0.99. This suggests that the model achieves a 99% accuracy rate when making predictions for instances belonging to either class 0 or class 1.

In addition, the concept of recall refers to the model's capacity to accurately detect and classify all occurrences of the positive class. In both instances, the recall rate is 0.99, suggesting that the model successfully catches nearly all occurrences of both class 0 and class 1 in its predictions.

The F1-score, which harmoniously combines precision and recall, also achieves an impressive 0.99 for both class 0 and class 1, signifying an excellent overall model performance.

Support provides insight into the number of instances within the test dataset for each class, revealing 7130 instances of class 0 and 6340 instances of class 1.

Overall accuracy on the test dataset is a remarkable 0.99, showcasing that the model correctly predicts the class for 99% of the test instances, underlining its high accuracy.

The "Macro Avg" row illustrates the average of precision, recall, and F1-score across both classes, with all metrics equalling 0.99. This indicates consistent and high-quality performance on average for the two classes.

Finally, "Weighted Avg" adjusts for class imbalance by assigning more weight to the class with a larger number of instances, yet all metrics remain steadfastly at 0.99. This underscores that the model maintains its exceptional performance even when factoring in the class distribution within the dataset.

In summary, this classification report underscores the exemplary performance of the binary classification model, demonstrating exceptional accuracy, precision, recall, and F1-score for both classes, and highlighting its effectiveness in identifying instances from class 0 and class 1 within the test data.

```
true_labels = y_test
# Calculate the confusion matrix
conf_matrix = confusion_matrix(true_labels, y_pred)

# Plot the confusion matrix using seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Purples', cbar=False)
plt.xlabel('Predicted_Labels')
plt.ylabel('True_Labels')
plt.title('Confusion Matrix')
plt.show()

plt.figure()
fpr, tpr, thresholds = roc_curve(true_labels, y_pred)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, color='darkgray', lw=3, label='ROC curve (area = %0.3f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='skyblue', lw=3, linestyle='--')
plt.xlim([0.0, 1.05])
plt.ylim([0.0, 1.07])
plt.xlabel('False Positive Rate(FPR)')
plt.ylabel('True Positive Rate(TPR)')
plt.title('Receiver Operating Characteristic(ROC)')
plt.legend(loc="lower right")
plt.show()
```

Figure 32 Code for plotting confusion matrix and ROC curve

The provided code sample is utilised to evaluate and visualize the effectiveness of a binary classification model through the examination of its confusion matrix and ROC curve.

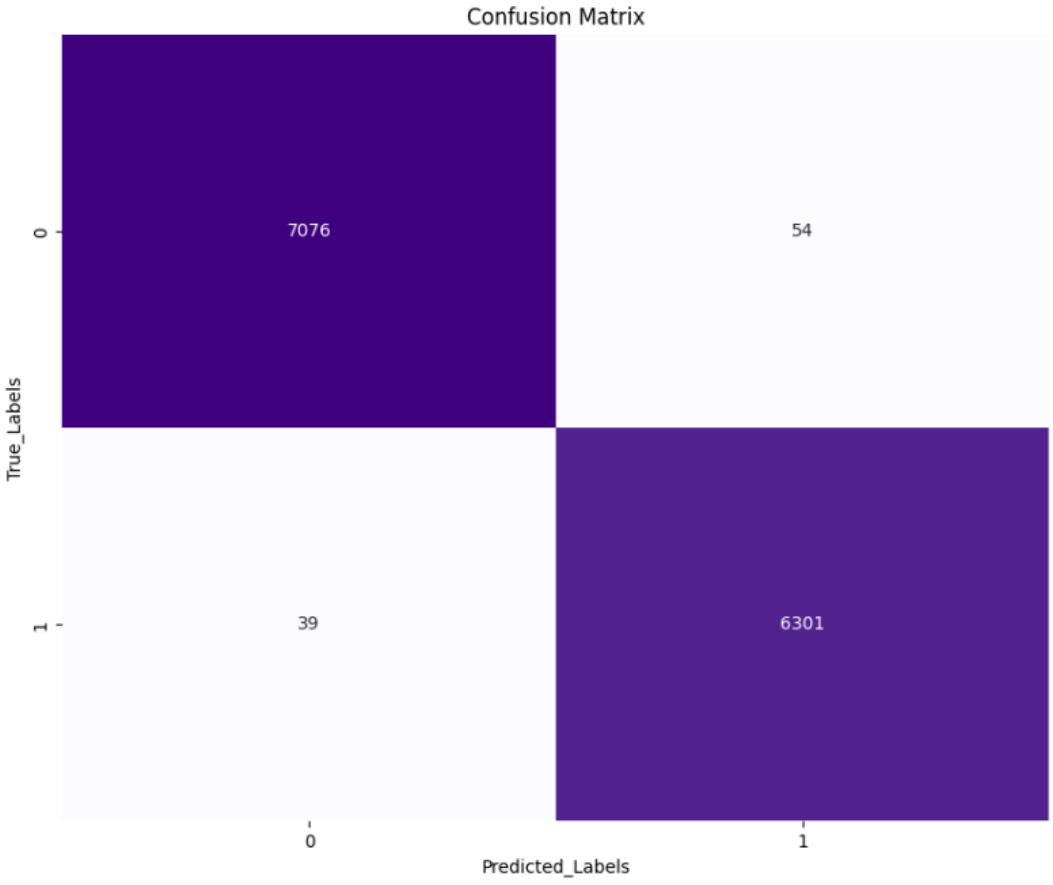

Figure 33 Confusion Matrix for Support Vector Machine

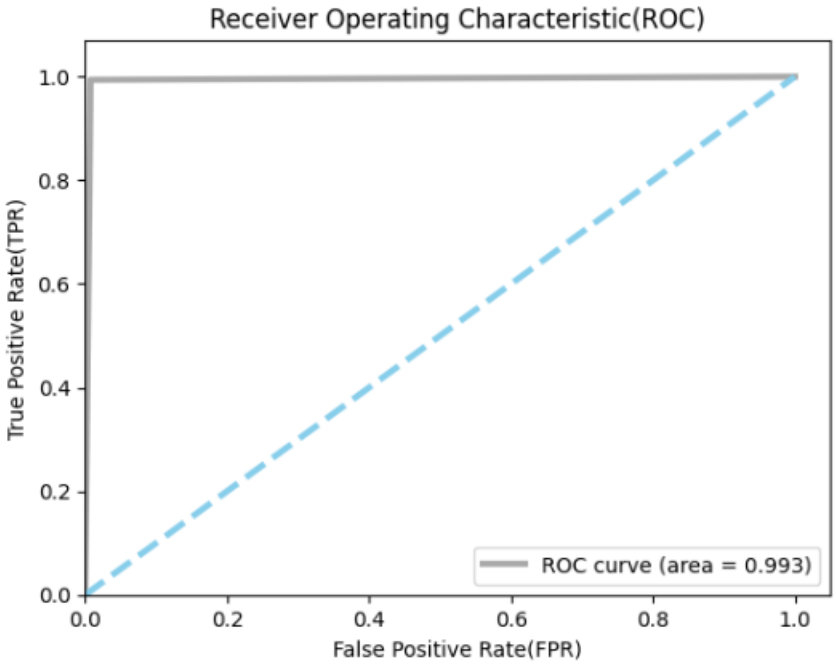

Figure 34 ROC curve for Support Vector Machine

Chapter V

5. Results and Findings:

The LR (Logistic Regression) and SVM (Support Vector Machine) methods were utilised in this study to detect instances of misinformation. The confusion matrix for the LR algorithm is presented in Figure 26. It shows that the true positive (TP) value, representing the correct prediction of fake news when fake news is delivered, is 7025. Additionally, the false positive (FP) value, indicating the incorrect prediction of true news when fake news is given, is 105. In the present context, Figure 33 provides a visual representation of the confusion matrix associated with the support vector machine (SVM) algorithm. Specifically, it displays the values for true positive (TP), which is recorded as 7076, and false positive (FP), which is recorded as 54. The SVM model exhibits higher accuracy compared to the LR model. The examination of the matrices indicates that Model 2, specifically Support Vector Machine (SVM), demonstrates superior performance compared to Model 1, which is Logistic Regression, in terms of accuracy, precision, recall, and F1-score. The support vector

machine (SVM) model exhibits a reduced occurrence of false positives and false negatives, hence resulting in improved performance ratings across the board. Nevertheless, the selection of these models can differ based on the specific demands of the application and the balance between precision and recall.

For the purposes of this study, Support Vector Machine (SVM) models were used to make predictions using the given dataset. The generated model achieved a level of accuracy of 99.309%.

Chapter VI

6. Conclusion:

In conclusion, the fake news is one of the high concerns around the world and it harms the many things (Society, people's health, economy of the country and many more) when the bogus news is circulating among the people. To detect fake news is one of the toughest things to do. So, I gathered two datasets i.e., fake, and true news datasets from Kaggle for the experiment purpose to see what's the outcome of the datasets once the datasets get trained and tested in this research paper.

This study aims to identify fraudulent news by utilising machine learning models such as Logistic Regression (LR) and Support Vector Machine (SVM), together with Natural Language Processing (NLP) approaches. Within the scope of this study, the investigation focuses on the utilisation of Logistic Regression and Support Vector Machine (SVM) algorithms as effective methodologies for the identification of false information, capitalising on their proficiency in examining textual material and extracting significant patterns.

Both the models have performed well enough to classify the false news but what we have obtained is support vector machine surpasses the competitor model i.e., logistic regression in term of precision. We find out that preprocessing the text data before feeding to the machine learning model is one of the most important steps while obtaining patterns from the text data.

To conclude, implementation of cleaning and obtaining ready to be fed text is one of the most crucial parts of large language processing which eventually help to achieve better classification.

7. Future Work:

In context with these encouraging findings, next investigations might go into possibilities for enhancing NLP-driven models in the realm of fraudulent news identification. This approach could involve implementing advanced feature engineering techniques, carefully choosing appropriate algorithms, and optimising parameters for support vector machine (SVM) models, all while ensuring the integrity of the effective natural language processing (NLP) prior treatment methods. Furthermore, enhancing the dataset by including more diverse selection of news items and integrating sophisticated natural language processing (NLP) techniques such as sentiment analysis and topic modelling may enhance the accuracy and practicality of the detection process.

Furthermore, the implementation of the Support Vector Machine (SVM) model in real-world scenarios, such as integrating it into browser extensions or news aggregator platforms, presents a promising direction for future investigation. The utilisation of NLP holds the potential to make an important impact in combating disinformation within the digital era, as it enables the transformation of our research outcomes into practical tools.

8. References

Alotaibi, F. . L. & Alhammad, M. M., 2022. Using a Rule-based Model to Detect Arabic Fake News Propagation during Covid-19. *International Journal of Advanced Computer Science and Applications,* 13(1).

N. Leela, S. R. K. & M., A., 2022. *Fake News Detection System using Logistic Regression and Compare Textual Property with Support Vector Machine Algorithm.* India, s.n.

Peng, J., Lee, K. L. & Ingersoll, G. M., 2002. An Introduction to Logistic Regression Analysis and Reporting. *The Journal of Educational Research,* Issue 10.1080/00220670209598786.

Ahmed H, T. I. S. S., 2018. *Detecting opinion spams and fake news using text.* [Online] Available at: https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/ISOT_Fake_News_Dataset_ReadMe.pdf

Anand, M. & Burra Venkata Durga, K., 2022. *Fake News Detection Using Deep Learning and Natural Language Processing.* Toronto, s.n.

Anon., n.d. [Online] Available at: https://www.ibm.com/topics/logistic-regression#:~:text=Resources-,What%20is%20logistic%20regression%3F,given%20dataset%20of%20independent%20variables

Arjun Roy, K. B. A. E. P. B., 2019. A Deep Ensemble Framework for Multi-Class Classification of Fake News. *Department of Computer Science and Engineering,.*

Galea, A., 2018. Beginning Data Science with Python and Jupyter. In: s.l.:Packt Publishing.

Iftikhar Ahmad, M. Y. S. Y. M. O. A., 2020. Fake News Detection Using Machine Learning Ensemble Methods. *Hindawi,* Volume 2020.

KattamuriMeghna, n.d. *Geeks for Geeks.* [Online] Available at: https://www.geeksforgeeks.org/python-introduction-matplotlib/

Leela Siva Rama Krishna , N. & Adimoolam , A., 2022. *Fake News Detection System using Logistic Regression and Compare Textual Property with Support Vector Machine Algorithm.* s.l., s.n.

McKinney, W., 2017. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython.. In: s.l.:O'Reilly Media.

Nicollas R. de Oliveira, D. S. V. M. D. M. F. M., 2020. IEEE Signal Processing Letters. *A Sensitive Stylistic Approach to Identify Fake News on Social Networking.*

NLTK, 2023. *Natural Language Toolkit.* [Online] Available at: https://www.nltk.org/ [Accessed 2023].

Noshin Nirvana Prachi, M. H. M. E. H. R. E. A. a. R. K., 2022. Journal of Advances in Information Technology. *Detection of Fake News Using Machine Learning and Natural Language Processing Algorithms.*

Pacheco, M., n.d. https://www.google.co.uk/books/edition/LEARN_EVERYTHING_ABOUT_python/xdvPEAAAQBAJ?hl=en&gbpv=0. In: *LEARN EVERYTHING ABOUT python.* s.l.:GAVEA LAB, p. 92.

Paper, D., 2019. Hands-on Scikit-Learn for Machine Learning Applications: Data Science Fundamentals with Python.. In: s.l.:Apress.

Pooja, D., 2023. Data Visualization with Python Exploring Matplotlib, Seaborn, and Bokeh for Interactive Visualizations . In: s.l.:Bpb Publications.

Reis, J. C. et al., 2019. Supervised Learning for Fake News Detection. *IEEE Intelligent Systems,* 34(2), pp. 76-81.

Rohan Chopra, A. E. M. N. A., 2019. https://www.google.co.uk/books/edition/Data_Science_with_Python/RYmkDwAAQBAJ?hl=en&gbpv=0. In: *Data Science with Python.* s.l.:Packt Publishing, p. 426.

Samrudhi Naik, A. P., 2021. Fake News Detection Using NLP. *International Journal for Research in Applied Science & Engineering Technology .*

Scholkopf, B. & Smola, A. J., 2018. Learning with Kernels: Support Vector Machines. In: s.l.:MIT Press.

Shaik, M. A. et al., 2023. *Fake News Detection using NLP.* India, s.n.

Shivani, N., sultana, N., Bhavani, P. & Shravani4, P., 2023. *Fake News Detection Using Logistic Regression.* [Online] Available at: https://ijaem.net/issue_dcp/Fake%20News%20Detection%20Using%20Logistic%20Regression.pdf

Singh, A. & Patidar, S., 2022. *A Survey on Fake News Detection Using Machine Learning.* India, s.n.

st Ashfia Jannat Keya, S. A. A. S. M. S. S. P. J. G. M. F. M., 2021. Fake News Detection Based on Deep Learning. *nternational Conference on Science & Contemporary Technologies (ICSCT).*

9. Appendices

Project Plan:

Table 1 Table of project plan

Task no.	Tasks	Start Date	End Date	Duration	Status
Task 1	Detailed Project Proposal (DPP)	14/06/2023	22/06/2023	8	Completed
Task 2	Research and Literature Review for: Algorithms, Data Modelling and Data sets	18/06/2023	07/07/2023	19	Completed
Task 3	Organising System Requirements	08/07/2023	10/07/2023	2	Completed
Task 4	Interim Progress Report	11/07/2023	18/07/2023	8	Completed

Task 5	Data collection and Preparation Exploration of the dataset, Data Preprocessing, Split of datasets	19/07/2023	02/08/2023	15	Completed
Task 6	Further Research on Model Selection, Implementation and Model Training	03/08/2023	24/08/2023	21	Completed
Task 7	Calculating Score, Testing Accuracy	25/08/2023	31/08/2023	6	Completed
Task 8	Documentation and Project Closure	01/09/2023	18/09/2023	16	Completed

Gantt Chart:

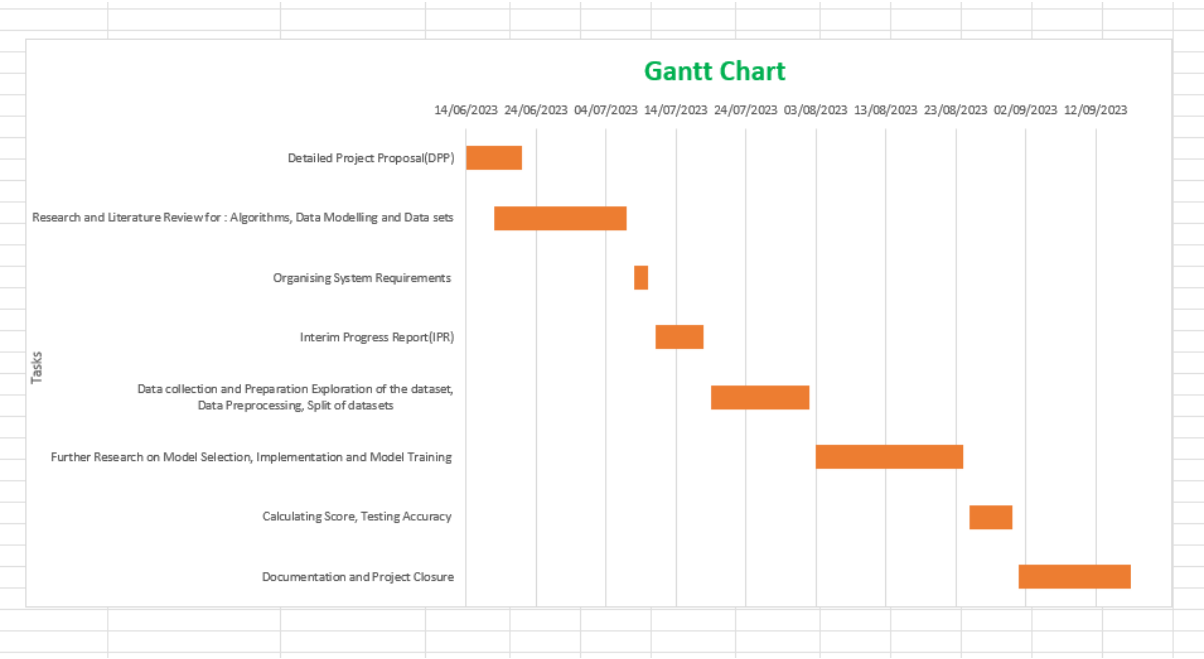

Figure 35 Gantt Chart

Code

```
#Importing Libraries

import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

import nltk

import tensorflow

from nltk.corpus import stopwords
```

```
from sklearn.utils import shuffle

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn.metrics import classification_report


import re

import string

df_fake = pd.read_csv('./Fake.csv/Fake.csv') #Fake dataset

df_true = pd.read_csv('./True.csv/True.csv') #True Dataset

df_true.columns

df_fake.columns

# we would like to make sure we have label for both true and false news

df_true.shape

df_fake.shape

df_fake['Label']=0

df_true['Label']=1

df_fake.shape[0]

df_true.shape[0]

# validation_set = pd.concat([df_fake[-10:],df_true[-10:]],axis=0)

# validation_set.reset_index(inplace=True)

df_fake.columns

df_fake.drop(['title','subject','date'],axis=1,inplace=True)

df_true.drop(['title','subject','date'],axis=1,inplace=True)

# to check if there is any null in dataset
```

```
df_true.isna().sum()

df_fake.isna().sum()


# now we are sure that there is no null values in both the attributes of the fake and true dataset

#we need to merge both the dataset together and suffle them in order to create randomness

df_true

merged_data = pd.concat([df_fake,df_true],axis=0).reset_index(drop=True)

merged_data.head(2)

nltk.download("stopwords")

stop_words = set(stopwords.words('english'))

def process_text(text):

  text=text.lower()

  text = re.sub('\w*\d\w*', '', text) #remove those words that contains digits

  text = re.sub('https?://\S+|www\.\S+', '', text)  #remove urls

  text = re.sub('\[/*?\]','',text) #remove text enclosed in sq bracket, this removes references if exists

  text = re.sub('<.*?>+', '', text) #Removes HTML tags

  text = re.sub("\\W"," ",text) #replace non-word characters with spaces

  #remove punctuation

  text = text.translate(str.maketrans('',string.punctuation))

  text = ' '.join(word for word in text.split() if word not in stop_words)

  #since we are more concerned on analyzing the textual content for classifying fake news , we ought to remove the hyperlinks on them.
```

```
  return text
merged_data['text']=merged_data['text'].apply(process_text)
merged_data.loc[111,'text']
# let's reshuffle the dataset we have concatenated the fake and real news(good practice)
merged_data = shuffle(merged_data,random_state=11).reset_index(drop=True)
import matplotlib.pyplot as plt

label_counts = merged_data['Label'].value_counts()

plt.figure(figsize=(8, 6))
ax = label_counts.plot(kind='bar', color=['green', 'red'])

#annotation is done for each bar with its count
for i, count in enumerate(label_counts):
    ax.text(i, count, str(count), ha='center', va='bottom', fontsize=8)

plt.title("Distribution of Labels (1: True, 0: Fake)")
plt.xlabel("Label")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()

merged_data['text_length'] = merged_data['text'].apply(len)
```

```
# true_mean = merged_data[merged_data['Label']==1]['text_length'].mean()

# true_std = merged_data[merged_data['Label']==1]['text_length'].std()

# fake_mean = merged_data[merged_data['Label']==0]['text_length'].mean()

# fake_std = merged_data[merged_data['Label']==0]['text_length'].std()

# true_synthetic = np.random.normal(true_mean, true_std, len(merged_data[merged_data['Label'] == 1]))

# fake_synthetic = np.random.normal(fake_mean, fake_std, len(merged_data[merged_data['Label'] == 0]))

range_of_x = (-100,20000)

true_length = merged_data[merged_data['Label']==1]['text_length']

fake_length = merged_data[merged_data['Label']==0]['text_length']

plt.figure(figsize=(5, 5))

plt.hist(true_length, bins=20, alpha=1, label='Fake', color='red',range=range_of_x)

plt.hist(fake_length, bins=20, alpha=0.3, label='True', color='yellow',range=range_of_x)

plt.title("Text Length Distribution by Label")

plt.xlabel("Text_Length")

plt.ylabel("Count")

plt.legend()

plt.show()

nltk.download("punkt")

from collections import Counter

from nltk.tokenize import word_tokenize

true_text = ' '.join(merged_data[merged_data['Label']==1]['text'])

fake_text = ' '.join(merged_data[merged_data['Label']==0]['text'])
```

```
true_word_freq = Counter(word_tokenize(true_text))
fake_word_freq = Counter(word_tokenize(fake_text))
# visualization of the most common words in fake and true news
common_words_true = true_word_freq.most_common(500)
common_words_false = fake_word_freq.most_common(500)
from wordcloud import WordCloud
words_t, frequencies_t = zip(*common_words_true)
words_f, frequencies_f = zip(*common_words_false)
true_word_freq_dict = dict(zip(words_t,frequencies_t))
fake_word_freq_dict = dict(zip(words_f,frequencies_f))
wordcloud = WordCloud(width=900, height=500,background_color='grey')
wordcloud.generate_from_frequencies(true_word_freq_dict)
plt.figure(figsize=(8,6))
plt.imshow(wordcloud,interpolation='bilinear')
plt.title("word cloud of common words in True news")
plt.axis('off')
plt.show
wordcloud.generate_from_frequencies(fake_word_freq_dict)
plt.figure(figsize=(8,6))
plt.imshow(wordcloud,interpolation='bilinear')
plt.title("word cloud of common words in Fake news")
plt.axis('off')
plt.show
#lets define the dependent and independent variables
```

```
X = merged_data["text"]
Y = merged_data["Label"]
#splitting the data into train test set
x_train, x_test, y_train, y_test = train_test_split(X,Y,test_size=0.3,random_state=11)
# conversin text to Vectors
from sklearn.feature_extraction.text import TfidfVectorizer
vector = TfidfVectorizer()
x_trainV = vector.fit_transform(x_train)
x_testV = vector.transform(x_test)
# implementation of logistic regression
from sklearn.linear_model import LogisticRegression
logistic_regression = LogisticRegression()
logistic_regression.fit(x_trainV,y_train)
# lets do prediction on test data
logistics_pred = logistic_regression.predict(x_testV)
logistic_regression.score(x_testV,y_test)
print(classification_report(y_test,logistics_pred))
# # to save the model to the file
# import joblib
# model_filename = 'logistic_regression.pkl'
# joblib.dump(LogisticRegression,model_filename)
# #load the model
# loaded_model = joblib.load('./logistic_regression.pkl')
# To test any string for fake news(try)
```

```
def ready(x):
    x=process_text(x)
    dicT={'Text':[x]}
    dfk = pd.DataFrame(dicT)
    v= dfk['Text']
    x_t_v = vector.transform(v)
    y_t_v = logistic_regression.predict(x_t_v)
    if y_t_v == 1:
        return "True News"
    else:
        return "fake news"
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, roc_curve, auc
true_labels = y_test
# Calculate the confusion matrix
conf_matrix = confusion_matrix(true_labels, logistic_regression.predict(x_testV))

# Plotting confusion matrix using seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Purples', cbar=False)
plt.xlabel('Predicted_Labels')
plt.ylabel('True_Labels')
plt.title('Confusion Matrix')
plt.show()
```

```
# Plotting ROC curve
plt.figure()
fpr, tpr, thresholds = roc_curve(true_labels, logistic_regression.predict_proba(x_testV)[:, 1])
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, color='darkgrey', lw=3, label='ROC curve (area = %0.3f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='skyblue', lw=3, linestyle='--')
plt.xlim([0.0, 1.05])
plt.ylim([0.0, 1.07])
plt.xlabel('False Positive Rate(FPR)')
plt.ylabel('True Positive Rate(TPR)')
plt.title('Receiver Operating Characteristic(ROC)')
plt.legend(loc="lower right")
plt.show()
#############support vector machine################
from sklearn.svm import SVC
svm_model = SVC(kernel='rbf',random_state=0)
svm_model.fit(x_trainV,y_train)



# import joblib
# model_filename = 'svm.pkl'
```

```
# joblib.dump(svm_model,model_filename)

# # #load the model

# # loaded_model = joblib.load('./logistic_regression.pkl')

y_pred = svm_model.predict(x_testV)

svm_accuracy = accuracy_score(y_test,y_pred)

svm_accuracy

print(classification_report(y_test,y_pred))

true_labels = y_test

# Calculate the confusion matrix

conf_matrix = confusion_matrix(true_labels, y_pred)


# Plot the confusion matrix using seaborn

plt.figure(figsize=(10, 8))

sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Purples', cbar=False)

plt.xlabel('Predicted_Labels')

plt.ylabel('True_Labels')

plt.title('Confusion Matrix')

plt.show()


plt.figure()

fpr, tpr, thresholds = roc_curve(true_labels, y_pred)

roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, color='darkgray', lw=3, label='ROC curve (area = %0.3f)' % roc_auc)

plt.plot([0, 1], [0, 1], color='skyblue', lw=3, linestyle='--')
```

```
plt.xlim([0.0, 1.05])
plt.ylim([0.0, 1.07])
plt.xlabel('False Positive Rate(FPR)')
plt.ylabel('True Positive Rate(TPR)')
plt.title('Receiver Operating Characteristic(ROC)')
plt.legend(loc="lower right")
plt.show()
```